Bachelor's Thesis Computing Science

Radboud University Nijmegen

Evaluating Local and Remote Storage Efficiency and Query Performance for CIFF-Based Inverted Indices

---

A Comparative Study Using DuckDB and Parquet Formats

Author:
Todd Bellavia
s1032723

First supervisor/assessor:
Prof. dr. ir. Arjen de Vries

Advisor:
Gijs Hendriksen

Second assessor:
Prof. dr. ir. Djoerd Hiemstra

January 15, 2025

Abstract

Efficient storage and querying of inverted index files is critical for modern search engines. This thesis evaluates the performance of CIFF-based storage of inverted indices in local and remote environments, addressing the trade-offs between storage efficiency, query performance, and scalability for distributed information retrieval systems.

The study compares local CIFF storage, optimized through DuckDB's compaction techniques, against remote storage utilizing Parquet files hosted on a Minio S3 bucket. Local storage consistently outperformed remote setups in query speed, with a 2.9 GB storage footprint post-compaction. Remote storage, though more scalable, required 6.2 GB and was significantly influenced by network conditions and hardware capabilities.

The findings emphasize that local storage is ideal for scenarios demanding low latency and high-speed querying, while remote storage's viability hinges on robust network infrastructure. Recommendations include strategies for optimizing partitioning and mitigating network overhead to enhance CIFF's applicability in distributed systems.

This research advances the understanding of CIFF's performance across storage environments, offering practical insights for implementing efficient, scalable information retrieval systems.

Contents

Chapter 1

Introduction

In the field of information retrieval, the storage and querying of inverted index files plays a crucial role in determining search engine performance [34]. As data scales, balancing storage efficiency and query performance becomes increasingly complex, particularly when considering different storage architectures and file formats. Locally stored CIFF (Common Index File Format) files offer a novel approach to standardizing inverted index structures, enabling replicable comparisons [14, 27]. Alternatively, Parquet files (a columnar storage format commonly used in distributed systems like Amazon S3) are optimized for scalability and efficient data access in cloud environments.

This thesis investigates the benefits and trade-offs between these two approaches by addressing the research question: *How do local storage and remote object storage compare in terms of storage efficiency for inverted index files, and what are their implications on querying performance?*

With the exponential growth of data in search engines and information retrieval systems, optimizing the storage of inverted index files while maintaining querying efficiency is critical. CIFF files, designed to standardize and streamline index structures, offer new types of system flexibility. When processed locally, index exchange does not negatively affect query performance, provided ample local storage is available [21]. Their application in larger datasets shows potential but remains underexplored. In contrast, remote storage solutions, such as Parquet files on Amazon S3, provide scalability for vast datasets but come with challenges, including network latency and increased storage overhead [25]. Understanding how these two approaches compare in storage efficiency and query performance will provide valuable insights for designing storage architectures in search engines and related projects.

Although CIFF files have emerged as a promising solution for standardizing inverted the data stored in index structures, their use in remote storage

environments remains largely untested. Meanwhile, Parquet files excel in scalability and analytics. CIFF nor Parquet have been evaluated for their possible usage in distributed IR query processing. These gaps highlight the need for further research into how CIFF performs in remote storage contexts and how it compares to cloud-optimized formats like Parquet.

The experiments conducted in this study revealed clear trade-offs between the two approaches. Local CIFF files, loaded into DuckDB, required approximately 2.9 GB of storage, while the remote Parquet files consumed 5.8 GiB (roughly 6.2 GB), including all partitions. Query performance also differed significantly: the local setup completed 100 random ORCAS queries in an average of roughly 340 seconds on Laptop 1 (one of the two laptops used during this study), whereas the remote setup took approximately 100 minutes for the same queries, on home wifi, on the same laptop. On the other hand, Laptop 2 approached similar times to the local average on Laptop 1 with its remote queries, using campus cable internet. These results underscore the strengths of local storage in delivering faster query response times due to reduced latency and direct access. However, the local approach is limited by scalability and hardware constraints, making it less suitable for handling very large datasets.

In contrast, the remote storage setup, despite slower query times due to network overhead and partitioned data access, offers scalability and global accessibility, which are critical for distributed systems or large-scale deployments. These findings emphasize the importance of balancing storage efficiency, scalability, and query performance in designing modern search engine architectures. By comparing these two approaches, this research provides actionable insights into when local or remote storage might be more appropriate, depending on application needs.

The remainder of this thesis is organized as follows, with each chapter building on the foundations of this research to provide a comprehensive understanding of the methodologies, results, and implications of comparing local and remote storage systems.

Preliminaries: This chapter provides foundational knowledge, introducing inverted indexing, CIFF files, and the challenges of comparing inverted indices across local and remote storage systems. It also outlines the concepts of storage efficiency and query performance.

Methodology: This chapter explains the research design, experimental setup, and data preparation processes, including ranking models, partitioning strategies, and measures to ensure data consistency.

Results: This chapter explains the research design, experimental setup, and data preparation processes, including ranking models, partitioning strategies, and measures to ensure data consistency.

Related Work: This chapter presents related work pertaining to this project, and the context in which it is placed from a top down view. It places CIFF in the context of remote storage in information retrieval and identifies gaps within that context, and highlights contributions that are made in this project.

Analysis and Discussion: This chapter interprets the results, examining trade-offs between storage efficiency and query performance, the influence of data flaws, recommendations, and practical implications of the findings.

Conclusions: This chapter summarizes the key findings, their implications for information retrieval, and future research or practical implementations.

Chapter 2

Preliminaries

The Preliminaries chapter serves as a foundational section that introduces the key concepts, technical background, and essential frameworks necessary to understand the research presented in the thesis.

2.1 Introduction to Inverted Indexing

The inverted index is a widely used data structure that enables efficient full-text search over large datasets, such as those found in information retrieval systems and search engines [34]. The goal of an inverted index is to store a mapping between terms and the documents they appear in, enabling quick identification of all documents containing a given term. This is achieved using postings lists, where each list corresponds to a term and contains entries, or postings, representing all the documents in which the term appears (as opposed to mapping one document to all the terms it contains, hence the name 'inverted index'). Each posting typically includes the document ID and may also store additional information such as term frequency (the number of times the term appears in the document) and positions (the locations of the term within the document). It is important to note that the structures mentioned previously may widely differ based on use cases and system needs [23]. For example, in this project, positional data is not used as the focus is on term frequency-based ranking, while document frequencies are central to BM25 calculations (a specific ranking algorithm), which are integral to the querying process.

In addition to the postings list, the inverted index's dictionary or vocabulary table maps actual terms to term IDs. A separate table or structure maps document IDs to the underlying document metadata, including the plain text. The inverted index itself is not responsible for storing the actual document content, however, it allows for fast retrieval of relevant document IDs. Once the relevant documents are identified, their content and meta-

data are accessed using the document IDs, ensuring efficient querying while keeping the search process independent of the document storage format.

2.2 Introduction to CIFF Files

Information retrieval scientists in the past have had a wide variety of options for query matching tools and ways to store inverted indices, both essential components of open-source search engines. As new technologies develop and older models are modernized, this variety will only expand. While having diverse query evaluation techniques and ranking methods is crucial for advancing efficiency and proficiency in information retrieval, this diversity introduces challenges. One key challenge is the difficulty of making fair and replicable comparisons among the different search engines, which handle inverted indices in unique ways. A promising solution is to standardize the format in which these inverted index structures are shared, and one such standard is the Common Index File Format (CIFF).

CIFF files serve as a common, standardized format for storing inverted index data, much like a universal cable adapter between differing systems. To illustrate an example use case of this file type, consider two scientists working on separate projects who wish to compare their distinct ranking algorithms. Although both may use the same document corpus, the way their respective search engines process and index these documents is likely to differ, potentially leading to results influenced by differences in index structures rather than the ranking algorithms themselves. CIFF files mitigate this issue by enabling the exchange of uniform datasets, preserving key index information such as posting lists, term frequencies, and document lengths. This ensures that comparisons are made on an equal footing, allowing researchers to focus on the effectiveness of their ranking algorithms, independent of underlying structural variations. In the context of this project, CIFF provides a standardized baseline for comparing storage efficiency and query performance across local and remote systems.

2.3 Challenges of Comparing Inverted Indices Across Different Systems

CIFF's standardized format for converting between two separate formats is more appealing than the alternative, which would require $n \times (n-1)$ conversions for n formats. Simply put, a wrapper for each possible format to convert to and from another (different) format. This quadratic growth in conversion pathways introduces unnecessary complexity, especially as the

number of formats increases, and does not offer any efficiency gains. Using CIFF as an intermediate format simplifies the process by providing a single, consistent structure for index exchange. Additionally, using CIFF introduces no query time performance penalties, as the underlying index structures remain unchanged, preserving the system's original performance characteristics [21].

However, CIFF is not without limitations. While it simplifies format conversions, it may introduce preprocessing overhead depending on how each system implements its index structures. Reaching widespread adoption of CIFF is also challenging, as many systems rely on proprietary formats that require custom read and write functionalities. Furthermore, CIFF files can be large, and distributing them over a network (particularly when handling sizable datasets) can be cumbersome, introducing practical challenges in real-world applications.

Another issue arises with interoperability between systems developed in different programming languages, such as Java and C++ [21]. The alternative approach of using wrappers to bridge these implementations introduces additional overhead, which can skew efficiency-focused comparisons, making it difficult to conduct fair performance evaluations. Despite the aforementioned challenges, CIFF remains a more viable solution than wrappers, particularly in research where the ability to conduct fair and replicable comparisons between different index structures is essential.

2.4 Local Storage vs. Remote Object Storage

Local storage and remote object storage represent two fundamental approaches to data management, each with unique benefits and trade-offs related to scalability, access speed, and storage efficiency.

Local storage refers to data housed on physical devices such as hard drives or SSDs, directly connected to the system querying the data. This close proximity offers low-latency access, making it ideal for applications requiring real-time performance. Local storage excels in environments where fast data retrieval and minimal overhead are critical. However, its scalability is limited by physical hardware constraints, meaning expansion often requires significant investment in infrastructure.

On the other hand, remote object storage, like cloud-based solutions such as Amazon S3, provides virtually unlimited scalability. Data is stored across distributed data centers, allowing for global accessibility. This model is ideal for managing large datasets across various locations. However, querying remote data introduces network latency, especially problematic for frequent

or real-time queries, due to the reliance on network transfers [19]. Remote storage is typically optimized for sequential data access, making it highly efficient for bulk retrieval but less suitable for random access queries, which often require retrieving large objects or files. As a result, choosing between local and remote storage depends heavily on the specific application's needs for scalability, latency, and query efficiency.

This comparison is central to this research, as it investigates the trade-offs between low-latency local storage for CIFF files and the scalability of remote Parquet files on distributed systems.

2.5 Storage Efficiency for Inverted Index Files

Storage efficiency refers to how effectively data is stored in relation to the space it occupies on disk or in memory and processing costs [31]. For inverted index files storage efficiency is a key factor that directly impacts both query performance and scalability. Efficient storage minimizes the physical footprint of index files without sacrificing the speed or accuracy of query operations.

Several factors influence the storage efficiency of inverted index files. Compression techniques and algorithms are commonly used to reduce the size of postings lists, term dictionaries, and other index components. Efficient compression can save significant storage space, but introduces a trade-off; the more complex the compression, the more processing power is needed to decompress the data during a query. Therefore, the optimal level of compression balances storage savings with acceptable query latency, depending on application. An example could be long-term data archival, such as storing large historical datasets or company log files that are queried infrequently. In these scenarios the focus is on minimizing storage costs rather than ensuring low-latency query responses.

Storage format plays a role in efficiency. Local storage solutions often allow for more customized storage optimizations, such as organizing data based on access patterns, which can lead to faster querying. On the other hand, in remote object storage, data is often stored as larger objects, which can reduce granularity and affect query efficiency, especially for random access operations.

For instance, this project leverages Snappy compression for Parquet files, a lightweight algorithm that optimizes storage space without significantly increasing query latency. The columnar format of Parquet further enhances compression by storing similar data together, reducing redundancy.

2.6 Querying Performance

Querying performance measures the efficiency of data retrieval processes in response to search queries. It forms a major part of the system latency (response time) and throughput (the number of queries handled over a period). In local storage systems, data resides on physical devices close to the querying machine, enabling low-latency access. This direct connection minimizes data transfer times and ensures high-speed retrieval, especially for random access patterns, where different parts of the dataset are frequently accessed. Consequently, local storage systems are well-suited for applications requiring real-time responses and high query frequencies.

In contrast, remote object storage systems handle data across distributed environments, accessed over a network. This setup introduces network latency and bandwidth limitations, which can slow down query performance, particularly for random access queries that require retrieving smaller, scattered pieces of data from large objects. While remote storage systems excel at bulk data retrieval due to their optimization for sequential access, they struggle with frequent, small-scale queries. Techniques such as caching can alleviate some of these performance issues by storing frequently accessed data on the querying device, thereby reducing the need for repeated data transfers and improving overall query speed.

Chapter 3

Related Work

This chapter provides a brief overview of existing research on inverted indexing, CIFF-based storage, and the performance trade-offs between local and remote storage systems. By situating this study within the broader field of information retrieval, the chapter highlights the gaps addressed by this research.

3.1 State of the Art in Inverted Indexing and CIFF

Inverted indexing is a cornerstone of information retrieval, enabling efficient querying of large datasets [34]. The Common Index File Format (CIFF) has emerged as a standardized solution for sharing and comparing inverted index data across systems, facilitating reproducibility and interoperability [21]. Prior work on CIFF has focused on its utility in local environments but offers limited insights into its performance when used in remote object storage systems [21].

3.2 Local vs. Remote Storage in Information Retrieval

Research on storage efficiency and query performance has traditionally compared local and cloud-based systems, emphasizing the trade-offs between latency and scalability [20]. Local storage is often associated with superior query performance due to its low-latency, direct access, while remote storage offers scalability and accessibility advantages at the cost of increased latency and network dependency [13]. However, a lack of comprehensive benchmarks for inverted file processing in information retrieval systems in these scenarios has limited the ability to generalize findings to real-world applications.

3.3 Identified Gaps and Contributions

While existing studies explore storage formats like Parquet for remote environments, little work has been done to assess CIFF's compatibility with cloud-based systems or to optimize its performance in such setups. Furthermore, strategies for partitioning data to align with diverse query patterns in these contexts remain underexplored.

This research addresses these gaps by providing a comparative evaluation of CIFF storage in local and remote environments, highlighting the impact of network conditions, hardware, and partitioning strategies on query performance. The findings contribute to a better understanding of how CIFF can be leveraged for scalable, efficient information retrieval systems.

Chapter 4

Methodology

All the code necessary to replicate the results of this project is publicly available on GitHub at `https://github.com/YeOldeOak/BScThesis`. Relevant snippets of the code are also provided in Appendix A, with references to specific sections of the thesis where they are discussed, enabling readers to follow and understand the methodology in greater detail.

4.1 Research Design

This research aims to evaluate the trade-offs between locally stored CIFF files and remotely stored Parquet files in terms of storage efficiency and query performance. The study is centered around the question: *How do local storage and remote object storage compare in terms of storage efficiency for inverted index files, and what are their implications on querying performance?*

To achieve this, the research was designed to control for variability across key factors, including query tools, file formats, and testing environments. DuckDB, a SQL-based analytical database, was selected as the querying engine due to its robust support for querying Parquet files on S3, and available load functionality for CIFF data [5, 11]. Using DuckDB ensures consistency in query execution mechanics, allowing observed differences to be attributed to the storage setups and remote querying optimizations rather than software variability.

The study used the ORCAS dataset, a real-world benchmark of anonymized Bing search queries, to test the system under realistic query workloads [4, 24]. This dataset includes both frequent and niche queries, ensuring that the evaluation covers a broad spectrum of search scenarios. The queries were tested on datasets derived from CIFF and Parquet formats, with the goal of comparing their impact on storage overhead, query execution time, and overall system performance.

Query performance was assessed under both local and remote storage setups. For local tests, CIFF files were stored on testing devices with comparable specifications, while remote Parquet files were hosted on a Minio S3 object storage system. To account for variations in network quality, remote tests were conducted under diverse conditions, including home Wi-Fi, cable connections, and high-speed campus networks. This diversity provides insights into how network performance influences remote query latency.

The research captures key performance metrics such as query latency (time required for result retrieval), throughput (queries processed per second), and storage overhead (file size and compression effects). In addition, profiling tools like DuckDB's `EXPLAIN ANALYZE` were used to differentiate between operator-specific timings (e.g., `Read_Parquet`) and overall query times [10]. These tools provided a detailed breakdown of query execution to identify bottlenecks and highlight opportunities for optimization.

By focusing on real-world workloads, consistent testing environments, and a robust set of metrics, this research aims to offer practical insights into the feasibility of using remote Parquet storage as an alternative to local CIFF storage for information retrieval systems.

4.2 Experimental Setup

4.2.1 System Architechture

The system architecture for this study was designed to enable a fair and reproducible comparison between locally stored CIFF files and remotely stored Parquet files. The experiments utilized two distinct environments: a local storage setup on two laptops and a remote storage setup leveraging a Minio S3 bucket. Care was taken to ensure consistent experimental conditions across all setups, with the same version of DuckDB used throughout to eliminate potential biases introduced by software differences. The hardware specifications of the testing devices and network conditions played a pivotal role in influencing the observed performance metrics.

Device Specifications

The experiments were conducted on two laptops, referred to as Laptop 1 and Laptop 2, whose specifications are summarized in Table 4.1. The decision to use two laptops instead of a single machine was driven by the need to ensure a fair, comprehensive, and robust evaluation of the environments. Both laptops featured SSDs and 16GB of RAM, providing sufficient hardware capacity for efficient local storage and querying.

Specification	Laptop 1	Laptop 2
CPU	Intel Core i5-8300H @ 2.30GHz	AMD Ryzen 7 4800H @ 2.90GHz
Cores / Threads	4 Cores / 8 Threads	8 Cores / 16 Threads
RAM	16GB SODIMM	16GB SODIMM
Storage	SSD	SSD
Operating System	Windows 11	Windows 11

Table 4.1: Device Specifications of Testing Laptops

Local Storage Setup

The local CIFF files were imported in a DuckDB database and queried directly on the laptops. The use of SSDs ensured high-speed sequential and random data access, minimizing potential bottlenecks [16]. Both laptops ran Windows 11, with the file system formatted as NTFS. Key considerations for the local setup included the absence of network dependency, eliminating latency as a factor in query performance and consistent performance across trials due to minimal variability in hardware or system conditions. The DuckDB database system was configured to store the CIFF data with its default "constant" compression and leveraged the laptops' SSDs for rapid query execution [9]. All queries were executed with identical DuckDB configurations on both laptops to ensure uniformity.

Remote Storage Setup

For the remote setup, the CIFF data was converted into Parquet files and stored on a Minio S3 bucket provided by the university. The Minio configuration included six active nodes for storage and processing, with an additional load-balancing node to distribute query requests evenly. The file system underlying the Minio bucket was based on XFS with a log block size of 32 MB. The network conditions varied across test locations, as summarized in Table 4.2. Connections ranged from home Wi-Fi to high-speed campus cable, which significantly influenced remote query performance. Notable network parameters included:

- **Home Wi-Fi:** Moderate bandwidth with higher latency, leading to slower remote query times.
- **Campus Cable:** Exceptional bandwidth and minimal latency, resulting in remote query times comparable to local storage under ideal conditions.

Network Condition	Upload (Mbps)	Download (Mbps)	Latency (ms)
Home Wi-Fi	13.2	24.4	44.1
Home ethernet	34.4	37.4	13.2
University Wi-Fi (Eduroam)	33.9-51.3	34.6-48.6	18.6
University Cable	112.0	120.0	5.2

Table 4.2: Network Conditions for Remote Query Tests

Fairness and Consistency in Testing

To ensure a fair comparison between the two storage setups, every effort was made to standardize the experimental conditions. The same version of DuckDB was consistently used across all tests to eliminate any discrepancies caused by software updates or optimizations. Laptop 2, despite having 16 threads, was limited to 8 threads to align with the capabilities of Laptop 1, ensuring a balanced comparison of hardware performance. Additionally, query timings for the remote setup carefully accounted for factors such as network latency, bandwidth fluctuations, and the operator-level breakdowns provided by DuckDB's `EXPLAIN ANALYZE` feature.

The connection to the Minio S3 bucket was established using either the university's science VPN in the case of Laptop 1, EduVPN in the case of Laptop 2 or, in the case of campus tests, directly through the Eduroam network. These differences in network infrastructure highlighted the significant impact of network quality on remote query performance, as detailed further in Chapter 5.

4.2.2 Data Formats

As mentioned in the preliminaries, CIFF (Common Index File Format) is a novel format designed to standardize data storage for fair and replicable comparisons of inverted indexes [21, 27]. CIFF structures data as a sequence of delimited Protobuf messages, ensuring consistency in how data is represented, which is essential for facilitating comparisons across different search engine implementations. CIFF files consist of three primary Protobuf messages: `Header`, `PostingsLists`, and `DocRecords`, arranged in that specific order.

- The `Header` contains metadata such as the number of `PostingsLists` and `DocRecords`, and the average document length.
- `PostingsList` messages represent the postings associated with search terms, where each list can contain one or more `Postings` (term-document mappings).

- `DocRecords` follow the `PostingsList` messages and provide metadata about individual documents, such as their identification numbers and lengths.

This structure is designed to efficiently serialize complex data in a reliable and portable manner, making CIFF ideal for supporting interoperability among diverse search engine systems [21, 27]. Further details on the design rationale can be found on the CIFF GitHub page [27].

For the purpose of this study, the CIFF files, initially designed for search engine indexing, were converted into a DuckDB-readable .db format using Apache Arrow. This conversion ensures that the data, originally structured for specialized inverted indexing, can be efficiently queried using SQL in DuckDB. The use of Arrow allows for a seamless transformation, making the data compatible with DuckDB's columnar storage format and enabling efficient querying of large datasets. The detailed steps of this conversion process are described in the Data Preparation section.

Parquet is a columnar storage format designed for distributed systems and optimized for big data analytics [17, 33]. Unlike row-based formats, Parquet stores data in columns, allowing for high compression rates and enabling the efficient querying of specific columns without needing to read the entire dataset [17, 33]. This feature is particularly advantageous in cloud environments, such as the Minio S3 bucket used in this experiment, where minimizing data transfer is crucial for maintaining query performance.

In addition to its cloud optimization, Parquet's columnar nature makes it an excellent fit for DuckDB, which is also built on a columnar storage model [6]. This synergy between Parquet's structure and DuckDB's query engine enhances both storage efficiency and query speed, particularly for analytical workloads. Parquet also supports several compression algorithms (including Snappy and Gzip) further improving storage efficiency by reducing the file sizes needed to store large amounts of data in a distributed system. This combination of efficient data storage, compression, and the ability to handle queries with minimal data transfer makes Parquet an ideal format for remote object storage in this study.

4.2.3 Querying Tools

Querying tools in this project are essential not only for executing queries but also for accurately evaluating the performance of these queries on remote Parquet files versus local CIFF files. The chosen tools were selected to ensure consistency in measurements across both storage systems. To achieve

compatibility between CIFF files and SQL-based querying, an external tool was used to convert CIFF files into a format readable by SQL databases [5].

DuckDB was chosen as the primary SQL-based querying tool because it supports Parquet natively, making it an ideal fit for this project's remote Parquet dataset [6]. Designed as an embedded analytics database, DuckDB is optimized for columnar storage formats like Parquet, which allows it to handle high-frequency queries and large analytical workloads effectively. Its compatibility with columnar formats was a primary factor in selecting it over alternatives such as CSV. Other columnar storage formats, like ORC, are discussed in later chapters.

DuckDB's columnar storage model and vectorized execution enhance its ability to process queries rapidly, especially with large datasets [29]. Additionally, DuckDB can perform file skipping and advanced query optimizations that reduce I/O overhead, making it well-suited for high-frequency analytical queries [29]. DuckDB's built-in benchmarking system, accessed via `EXPLAIN ANALYZE`, was utilized during experiments to measure exact query performance, providing detailed insights into execution plans and resource usage [10]. However, for the wall time results, Python's `time` module was used to measure query times overall, as some sections of the code could not accommodate the simultaneous use of `EXPLAIN ANALYZE` and value retrieval, and wouldn't account for Python overhead.

This project used DuckDB version 1.1.3, which introduced performance improvements that positively impacted query speeds. It is important to note that results may differ for versions prior to 1.1.3 due to the query execution optimizations introduced in this release. These improvements made DuckDB even more effective for handling complex queries in both local and remote storage environments. Future versions of DuckDB may improve upon this and produce even better benchmarks as a result.

DuckDB's compatibility with columnar storage, in-memory processing for efficient local querying, and its simple deployment as an embedded system make it especially suitable for research-focused applications [6, 29]. Other SQL-based databases, such as SQLite [15] and Postgres [1], were considered but ultimately not selected due to these specific advantages. DuckDB's feature set provided a more robust framework for a consistent comparison between local and remote storage systems.

[1] https://www.postgresql.org/

4.3 Data Preparation

4.3.1 Overview of Data Preparation Process

The primary goal of data preparation in this project is to establish a consistent and comparable dataset across CIFF files imported locally and remote Parquet files in cloud object storage (S3). This ensures that the results of storage efficiency and query performance tests are both reliable and unbiased, allowing for a fair evaluation of each storage system's capabilities.

The data preparation process involves several key steps, including configuring a custom BM25 ranking model for query evaluation, modeled after DuckDB's full-text search extension. This ranking model was selected to provide a consistent basis for relevance scoring across both datasets. Additional steps included partitioning for optimized remote storage access, and ensuring data consistency between formats. Compression and optimization techniques were applied automatically via DuckDB, streamlining storage and improving query efficiency.

Each of these steps, excluding partitioning for local CIFF, was implemented to maintain consistency in the dataset across local and remote environments, so that observed performance differences could be attributed to the storage systems and to the querying themselves rather than to any variations in data preparation.

The following sections provide a detailed breakdown of the data preparation process, covering the setup of the ranking model, dataset descriptions, partitioning strategies, and measures taken to ensure data consistency and optimization. Collectively, these steps ensure that the data used in this research is prepared to yield accurate and fair comparisons between local and remote storage environments.

4.3.2 Conversion to DuckDB

To enable efficient querying and analysis, the CIFF files were converted into a format compatible with DuckDB. This conversion process leveraged the CIFF2DuckDB tool, which uses the PyArrow library to facilitate data transformation. The Python script `ciff-arrow.py` from the **CIFF2DuckDB GitHub repository** served as the primary mechanism for this process. The conversion was straightforward:

- The path to the zipped CIFF index file was provided as input to the script.
- PyArrow was used to decompress and parse the CIFF data into an Arrow-compatible format.

- The resulting data was written into a DuckDB-readable `.db` file, preserving the postings data, term frequencies, and document metadata for further querying.

The PyArrow library's role was essential in ensuring an efficient and reliable conversion, as it supports in-memory columnar storage and seamless data transformation workflows. This step standardized the data for compatibility with DuckDB, which was subsequently used for querying both the local and remote datasets.

For access to the complete conversion code and implementation details, the script and repository can be found in the footnotes of this page [2].

4.3.3 Ranking Model

BM25 is a probabilistic ranking model within the Okapi Best-Match family of algorithms, widely used in information retrieval to score and rank documents based on term relevance. It is particularly effective in search engine applications, as it scores documents by estimating the probability of a document's relevance to a given query [2]. The full implementation code for the BM25 ranking model configuration and query process is provided in Appendix A

BM25 was chosen as the ranking model due to its well-rounded performance as a relevance ranking algorithm in information retrieval. Known for balancing simplicity with effectiveness, BM25 is widely adopted across various search systems for its robust handling of term frequency and document length, producing reliable relevance scores with minimal tuning [3]. This flexibility makes it a fitting choice for the project, where query evaluation is essential but not the primary research focus. BM25 provides reliable, consistent ranking performance, enabling fair comparisons of query efficiency across storage formats without requiring extensive customization [30].

The BM25 ranking model utilizes the well-known probabilistic formula, which calculates relevance based on term frequency (`tf`), inverse document frequency (`idf`), and two key parameters, `k1` and `b`. In this implementation, `k1` controls the sensitivity to term frequency, while `b` adjusts the normalization based on document length. To align the BM25 model with the specific characteristics of the dataset, parameters `k1` and `b` were fine-tuned. Initial experiments with several `k1` values (1.2, 1.5, and 1.8) revealed that `k1` = 1.5 provided a balanced ranking score across both short and long documents, while `b` = 0.75 appropriately normalized based on document length. These parameters were hard-coded into the BM25 implementation to ensure consistent application across all query evaluations, shown in the following code

[2] https://github.com/arjenpdevries/CIFF2DuckDB

block. DuckDB's full-text search also utilizes the BM25 ranking model, their implementation contains a `k1` value initialized to 1.2, which motivated initial testing with that value as well [7].

```
LOG((SELECT num_docs FROM stats) / (1 + q.df)) *
(p.tf * (1.5 + 1)) /
(p.tf + 1.5 * (1 - 0.75 + 0.75 * (d.len / (SELECT
   avgdl FROM stats)))) AS bm25termscore
```

BM25 was integrated into DuckDB's querying process by implementing a custom ranking function that dynamically applied the BM25 formula during query execution. This integration allowed for relevance-based ranking of both CIFF and Parquet datasets, providing a standardized method of evaluating query performance. The custom SQL queries for both local and remote setups, and the full BM25 implementation contained within these are detailed in Appendix A, demonstrating how the ranking model was incorporated into DuckDB's SQL environment for efficient execution. Alternatively the project's **Github page** can also be viewed to find the full code. The model's effectiveness in ranking relevance across varied document lengths is well-suited to this project's requirements, as it supports consistent and unbiased evaluations across the two storage formats. However, BM25's reliance on specific parameter settings could introduce sensitivity to certain query types. The code, as implemented, allows for these settings to be modified if further optimization is required.

4.3.4 Datasets' Descriptions and Sources

In this project, two main datasets are central to evaluating storage and querying performance: the CIFF dataset, which contains posting data and relevant metadata for querying, and the ORCAS queries dataset, which provides a collection of random, anonymized real queries. The ORCAS dataset serves as the query source, allowing for a randomized sample that forms the basis for testing against both local and remote storage systems [4]. The CIFF data, pre-assembled and indexed with Open Web Search's OWLER web crawler, includes posting lists and term metadata [26].

The CIFF dataset is suitable for this project as it contains approximately one billion postings, providing enough data to measure variations in storage size and query performance. It corresponds to one day of indexing from the Open Web Search project. The data's integrity and relevance are ensured by the OWLER crawler, which indexes diverse web content and prepares it in a standardized format. Meanwhile, the ORCAS dataset is ideal for this project as it is a large collection of anonymized, real-world queries [4].

This randomization and anonymization provide an unbiased basis for testing, with queries reflecting typical language and user intent. The ORCAS dataset was downloaded from Microsoft's official site [24]. The download option for the queries is the second one within the downloads table, under the name "orcas-doctrain-queries.tsv.gz". No changes were made to the file formatting or structure post-download, only when pre-processing for querying.

The OWLER web crawler's indexing process ensures that the CIFF data includes a substantial variety of terms. Specifically, the postings data comprises 12.2 million unique terms, with a broad distribution of both frequently and rarely used words. The metadata includes document length information, with lengths ranging from 0 to 370 thousand words, ensuring variability that supports comprehensive testing.

In summary, these datasets provide a robust basis for testing storage and querying performance, with the CIFF dataset offering a comprehensive index structure and the ORCAS dataset providing realistic, anonymized query data. The size and scope of these datasets were selected to balance practical considerations and to ensure relevance to real-world information retrieval tasks. Future research could expand on this project by exploring datasets in other languages or additional domain-specific content to assess generalizability. All data were reviewed for consistency and compatibility, ensuring that they accurately support the study's storage and querying goals.

4.3.5 Partitioning and Storage Setup

Partitioning is essential for organizing data in smaller, manageable sections, which optimizes both storage and query efficiency [22]. In this project, partitioning primarily serves the remote storage setup to improve data retrieval times, as querying large amounts of remotely stored data is inefficient without selective data access. By partitioning the dataset based on term IDs, each term is stored within a specific bucket, allowing targeted searches within a single bucket rather than across the entire dataset.

The partitioning strategy used for the remote dataset involves adding a new column, `thash`, to the postings data, which stores hashes of term IDs. These hashes were calculated by performing the modulo operation on the hashed value of the term ID by the desired number of partitions. Several partition sizes (including 25, 50, and 100) were tested to determine their impact on query efficiency. Testing showed a configuration of 40 partitions as optimal, yielding the best balance between query response times and data management complexity. The size of each partition was between 125 and 150 MiB with 40 partitions. This approach ensures that each partition contains terms with unique identifiers, facilitating faster access to relevant data during query execution.

In addition to partitioning the postings data, the dictionary table was also partitioned in the remote setup to improve query efficiency. Unlike the postings data, the dictionary was partitioned based on ranges of strings, using the first character of each term. For example, terms starting with 'a' might be grouped into one partition, while terms starting with 'b' would form another. This partitioning approach allows for efficient lookups when querying specific terms, as only the relevant partition needs to be accessed. Below is a code snippet illustrating how this partitioning was implemented by creating a macro to apply to the remotely stored dictionary:

```
CREATE OR REPLACE MACRO cr(term) AS
    ascii(term[1]) // 10;
COPY (SELECT term, termid, df, cr(term) AS
   crange FROM dict ORDER BY crange)
 TO 's3://user/pdict.parquet' (FORMAT PARQUET,
    OVERWRITE_OR_IGNORE);
```

Partitioning was only applied to the remote setup, as this research focuses on comparing CIFF files' storage and query performance locally against partitioned data stored remotely. The effects of partitioning local CIFF data are left for future work.

For the local storage setup, the CIFF data was stored without partitioning or additional compression to maintain the file in its original structure. The data was saved in a single `.db` file on an SSD for optimal access speed, allowing straightforward file path management for querying. Additional details on the local system architecture are provided in the System Architecture chapter. The remote storage setup involved separating the CIFF data into three main Parquet files: one for document metadata (e.g., document lengths and ID mappings), one for term-to-ID mappings, and one for overall statistics. The postings data partitions are stored alongside these files, each in its own Parquet file generated using DuckDB's Parquet writing function, which applies Snappy compression to optimize storage efficiency.

Partitioning can significantly impact query performance by enabling selective data access, which reduces the amount of data transferred and processed during a query. For instance, accessing a specific term ID or document ID within the remote setup can target the relevant partition directly, leading to faster query response times compared to querying an unpartitioned dataset. Additionally, storing postings data as partitioned Parquet files with Snappy compression can improve storage efficiency and may optimize I/O performance, as DuckDB can leverage the columnar format to skip irrelevant data during query execution. On the local setup, the CIFF file's structure

in a single .db file minimizes the complexity of data management and access, maximizing performance for local queries but limiting configurability. This setup allows for efficient sequential access to data, whereas the remote setup leverages partitioning and compression to minimize network latency and data retrieval costs.

Despite the benefits of partitioning for remote storage, there are some limitations to this approach. Partitioning can introduce increased management complexity as more partitions require additional handling and can lead to slight performance overhead if partitions are unevenly distributed or too numerous. Furthermore, the absence of caching in the remote setup ensures a straightforward comparison, but caching could have further optimized query performance by reducing network calls for frequently accessed data. The lack of partitioning in the local CIFF setup also limits comparisons of partitioned data between local and remote storage configurations, leaving this aspect for future research.

4.3.6 Data Consistency Measures

Ensuring data consistency is crucial in information retrieval projects, as flaws in the dataset can significantly impact both storage efficiency and query performance. In this project, data consistency checks were conducted to identify and address critical issues, ensuring a more accurate and uniform dataset for both local and remote setups.

Identified Data Flaws (in the postings dataset):

The following key issues were identified in the initial dataset:

1. **Non-Unique Document Identifiers:**
   The dataset initially contained instances where multiple captures of a document shared the same document identifier (`docid`). This violates the assumption of unique identifiers for each document and leads to incorrect document representations. For example, different captures of the same document would be treated as a single document, regardless of whether their content or metadata (e.g., length) differed. Resolving this issue was essential to ensure the validity of results and accurate term-document relationships.

2. **Duplicate Postings:**
   Duplicate entries were present in the postings table, where the same term-document pair (`termid`, `docid`) appeared multiple times. These duplicates inflated the term frequency (`tf`) values for affected terms and skewed the document frequency (`df`) statistics in the dictionary. Left unaddressed, these duplicates could bias ranking models, such as BM25, by disproportionately favoring redundant entries.

3. **Zero-Length Documents:**
   Approximately 3.9% of the dataset (roughly 120,000 documents out of 3.1 million) contained fields with zero length and no content. These entries, often caused by crawl errors or incomplete indexing, minimally impact storage size but will not affect query relevance as these documents never match terms.

Consistency Handling and Impact on Processing:

The identified issues were addressed through targeted corrections to ensure a consistent and valid dataset for experimentation:

1. **Fixing Non-Unique Document Identifiers:**
   To address the issue of non-unique `docid` values, each document identifier was made unique. In cases where multiple captures of a document shared the same identifier, the representation with the maximum document length (`len`) was retained. This choice avoided zero-length entries and ensured that each document had a distinct and meaningful representation in the dataset.

2. **Removing Duplicate Postings:**
   Duplicate term-document pairs in the postings table were removed by retaining only one occurrence of each unique `termid-docid` pair. The document frequency (`df`) for each term in the dictionary table was then recalculated to reflect the corrected postings. This ensured that term-document relationships were accurate and prevented inflated frequencies from affecting ranking algorithms.

3. **Zero-Length Documents:**
   Zero-length documents were retained in the dataset to maintain realism and simulate real-world imperfections often encountered in large-scale datasets. These entries were not excluded during corrections, as they provide a realistic test case for handling noisy data in information retrieval systems.

After these steps, the dataset was rechecked to confirm the absence of non-unique docid values and duplicate postings, ensuring consistency across both local and remote setups. The implications and impacts of these changes are discussed further in Chapter 6.

4.3.7 Compression and Optimization Techniques

Optimizing storage and querying performance is essential in this project, especially given the mixed local and remote storage setups. To this end, several compression and optimization techniques were implemented, focusing

on reducing storage size, improving data retrieval speeds, and maximizing computational efficiency. Key optimizations include Snappy compression, multi-core processing in DuckDB, and considerations for the remote environment's file system and I/O constraints.

The Parquet files generated for this project use Snappy compression, a lightweight algorithm optimized for high-speed compression and decompression. Snappy strikes a balance between compressing data to reduce storage footprint and maintaining fast access speeds, making it especially suitable for this project's query-intensive environment. With Snappy applied to the partitioned Parquet files on the remote Minio setup, data transfer times and storage requirements are attempted to be minimized. This compression strategy is advantageous for reducing network load, which is crucial in the remote setup where bandwidth is shared and I/O is limited. Although Snappy offers a lower compression ratio than algorithms like Gzip, its speed makes it more appropriate in cases where both storage efficiency and processing time are critical factors.

DuckDB leverages multi-core processing by default, utilizing all available CPU cores to parallelize query execution. This approach is beneficial for both local and remote querying, as it allows DuckDB to handle larger workloads and improves response times by executing multiple operations concurrently. For instance, when accessing the CIFF data on the local SSD, multi-core processing ensures rapid sequential access, whereas in the remote setup, it potentially helps mitigate the impact of network latency by retrieving data from multiple Minio nodes simultaneously.

In this project, DuckDB version 1.1.3 was utilized, benefiting from performance enhancements that improve query efficiency, particularly with filter pushdown optimizations for OR/IN conditions on integer columns. These updates allow DuckDB to apply zone map filters, reducing the number of unnecessary row groups scanned and improving execution times for ordered or densely populated columns. However, the specific

```
q.termid IN [...]
```

query pattern used in this project did not see direct performance improvements because the feature is not yet fully supported for this case. To address this, an alternative approach using UNIONs in DuckDB was implemented, which provided the necessary functionality while maintaining efficient query execution. These enhancements and workarounds collectively ensured that query operations remained lightweight and that performance remained consistent across both local and remote storage setups.

The remote storage setup runs on an XFS file system, configured with a log block size of 32 MB, which is designed to handle high-performance, multi-threaded workloads. XFS is also the filesystem recommended by MinIO, and

this recommendation likely stems from its advantages in managing concurrent access patterns, particularly in multi-user environments like the MinIO setup, where files are frequently accessed and updated by different processes. While Snappy compression reduces the amount of data transferred per query, DuckDB's multi-threading actually increases the network load by downloading more Parquet blocks concurrently. However, this parallelism ensures that queries are executed faster by fully utilizing available bandwidth and computational resources. An additional node functions as a load balancer, distributing query requests across the six MinIO nodes. While the load balancer centralizes network traffic, which could theoretically create a bottleneck or single point of failure, its role of simply forwarding traffic is computationally lighter compared to the data-serving tasks handled by the MinIO nodes. As such, it is unlikely to become the primary bottleneck, though this remains a hypothesis without concrete measurements.

Snappy compression reduces the data footprint, specifically in the remote setup, conserving storage space while maintaining fast decompression speeds during query execution. Meanwhile, DuckDB's multi-core processing capabilities, combined with the XFS file system's efficient I/O management, ensure that queries are executed quickly, even when dealing with high network loads and distributed data retrieval. These configurations also allow DuckDB to make optimal use of Parquet's columnar structure, enabling file skipping for more efficient I/O operations and reducing query response times [32]. Together, these technologies work to mitigate the inherent challenges of remote querying while maintaining efficient system performance.

In summary, these compression and optimization techniques ensure that the project's storage and querying goals are met efficiently, both locally and remotely, while maximizing system resource utilization.

Chapter 5

Results

5.1 Storage Efficiency

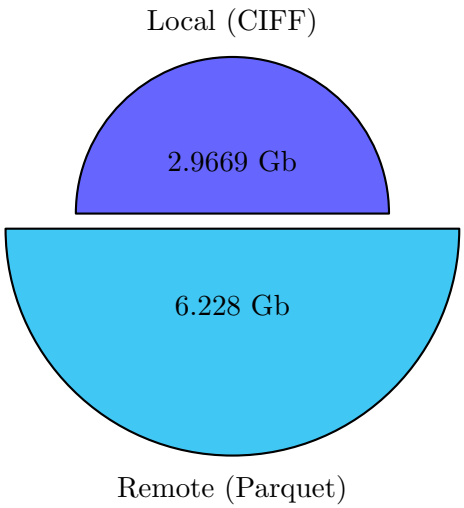
Storage efficiency refers to the ability to store data in a compact and optimized manner, balancing physical space usage with query performance. This section presents the storage results of importing local CIFF files into DuckDB databases and remote Parquet files, providing a detailed comparison of their space usage and discussing the impact of deduplication, compression techniques, and partitioning strategies. Additionally, we explore the scalability of remote storage solutions and consider alternative storage methods.

5.1.1 Storage Results and Comparison

The measured storage sizes for both setups are summarized in Table 4.1, the following pie chart visualizes the difference between the two.

Table 5.1: Comparison of Storage Efficiency Between Local CIFF and Remote Parquet Files

Storage System	File Type	Size (GB)	Compression Method	Aditional Details
Local (CIFF)	DuckDB .db	2.9669	Default "Constant" (and Compaction)	Space reclaimed after deduplication
Remote (Parquet)	Partitioned Parquet	6.228	Snappy (Default)	40 partitions for postings + metadata



5.1.2 Local CIFF Storage

The local storage setup used DuckDB to store the CIFF data as a single `.db` file. Initially, after deduplication, the database occupied approximately **4.8 GB**. However, DuckDB's space reclamation method was applied to eliminate unused space and compact the database, reducing its size to **2.9 GB** [8]. This process preserved the data integrity while optimizing its physical footprint without applying any additional compression.

Although DuckDB supports advanced compression techniques, such as ZSTD and GZIP, these were intentionally not utilized in this project to maintain a baseline comparison. Compression methods like ZSTD can achieve higher storage reductions but come at the cost of additional CPU overhead during decompression [18]. By avoiding these alternatives, the local setup remained straightforward and unoptimized for compression, providing a clean comparison against the default compressed Parquet files.

5.1.3 Remote Parquet Storage

The remote storage setup stored the CIFF data in partitioned Parquet files on a MinIO S3 bucket. Each Parquet file applied DuckDB's default Snappy compression, which strikes a balance between storage size and decompression speed. The postings data was split into 40 partitions, determined through initial testing of configurations with 25, 50, and 100 partitions. The 40-partition structure provided the best query performance without introducing excessive storage overhead.

While Snappy compression reduced the size of each Parquet file, the overall storage footprint of the remote setup remained significantly larger,

5.8 GiB (6.228 GB), nearly double the local CIFF database. Part of this difference can be explained by two factors. Firstly, by each partition introducing additional metadata, headers and file structures. This partitioning overhead in turn increases overall space usage. Secondly, by the columnar storage metadata. Parquet files store schema and block-level metadata for optimized querying, which likely adds to the file size. These factors are likely not the only aspects that attribute to such an increase in storage space.

5.1.4 Alternative Compression and Storage Methods

DuckDB's compatibility with Parquet was a critical factor for this project; however, alternative compression methods and storage formats could yield different results:

1. **Compression Techniques**

   - **ZSTD:**
     Achieves higher compression ratios but requires additional CPU resources for decompression, which can impact query performance.
   - **GZIP:**
     Provides strong compression but has slower read speeds, making it less suitable for high-frequency querying workloads.

   In this project, Snappy was chosen due to its lightweight compression and faster decompression speeds, ensuring consistent query performance. It being default in DuckDB also allows for easier replication.

2. **Storage Formats**

   - **ORC (Optimized Row Columnar):**
     ORC is a robust columnar format that supports advanced compression and indexing features [1]. However, DuckDB lacks built-in support for ORC, as confirmed in community discussions [12]. Implementing ORC functionality would have required custom solutions, which were beyond the scope of this project.

   Parquet's native integration with DuckDB made it the sensible choice, simplifying data processing while leveraging columnar storage benefits.

5.1.5 Scalability and Practical Implications

The scalability of the two setups highlights the trade-offs between storage efficiency and system flexibility:

- **Local Storage (CIFF):**
  - Compact and efficient, with a reduced size of 2.9 GB post deduplication and compaction.
  - Limited by hardware constraints, such as SSD capacity and computer resources, making it less suitable for handling very large datasets.
- **Remote Storage (Parquet on MinIO):**
  - Scalable and accessible, capable of handling much larger datasets across distributed nodes.
  - The use of a 6-node MinIO setup with a load balancer allows for efficient parallel access, making it ideal for distributed environments.
  - However, scalability comes at the cost of increased storage overhead and higher network latency for queries.

Future work could explore further optimizations, such as integrating caching mechanisms to reduce network access delays or testing alternative storage formats like ORC with custom implementations.

5.1.6 Conclusion

The local CIFF files demonstrated superior storage efficiency, occupying only 2.9 GB after deduplication and compaction. In contrast, the remote Parquet setup consumed 6.228 GB, primarily due to partitioning and metadata overhead, despite Snappy compression. While local storage offers compact efficiency, it is limited in scalability. Remote storage, on the other hand, provides virtually unlimited scalability and flexibility but incurs additional storage and network costs.

These results highlight the trade-offs between compactness and scalability, laying the foundation for further exploration of optimized storage techniques and formats in distributed information retrieval systems.

5.2 Query Performance

Efficient query performance is critical for evaluating the feasibility of remote storage solutions compared to local storage, particularly when handling large-scale datasets in information retrieval systems. This section investigates the factors influencing query performance, focusing on the interplay between hardware configurations, network conditions, and the underlying

storage architecture. By analyzing these aspects, this chapter provides insights into the practical trade-offs of using local CIFF storage versus remote Parquet storage.

The experiments leverage two distinct hardware setups (Laptop 1 and Laptop 2) and multiple network conditions to comprehensively examine query execution times across local and remote implementations. The results highlight the effects of varying bandwidths, latencies, and computational resources, showcasing the scalability and viability of remote object storage under optimized configurations.

Key performance metrics include the total query execution time, the time spent on reading Parquet files during remote queries, and their correlation with network speeds and connection types. This chapter also introduces comparisons between local and remote setups, presenting scenarios where remote storage approaches local performance, particularly with high-speed network connections and optimized hardware. By evaluating these results, this section establishes a firm understanding of the trade-offs and benefits inherent to both storage solutions.

Finally, this section lays the groundwork for recommendations on future improvements, such as partition size optimization and adaptive query strategies, to maximize the efficiency of remote storage solutions. This analysis serves as a stepping stone toward identifying best practices for integrating remote storage into modern information retrieval systems.

5.2.1 Local vs. Remote Query Performance Across Networks

This section evaluates the query performance of local CIFF storage in DuckDB and remote Parquet storage, focusing on the influence of network conditions. By isolating the role of network speed and latency, we aim to provide insights into the trade-offs between local and remote storage solutions for query execution.

Local Query Performance

Local CIFF storage demonstrated consistent and predictable query performance across all tests. For 50-query sample sizes, average query times ranged between 4.10 and 4.28 seconds per query (as shown in Table A.1) for Laptop 1. These results highlight the advantages of local storage, including direct access to data without network overhead and minimal variability in execution times.

Remote Query Performance

Remote Parquet storage exhibited higher variability in query times, heavily influenced by network conditions. Table A.1 from Appendix A shows that remote query times using Laptop 2 on slower connections, such as home Wi-Fi, ranged from 19.5 to 20.2 seconds per query for 50-query samples. In contrast, on high-speed campus cable, query times were reduced to an average of 4.5 seconds for 50-query samples, nearly matching local performance on Laptop 1.

These differences are further illustrated in Figures 5.1 and 5.2, where the performance of Laptop 2 under various network conditions is compared. Notably, campus cable consistently yielded the fastest results, demonstrating that remote storage can achieve performance on par with local storage when paired with optimal network infrastructure, in comparison to the local times of Laptop 1.

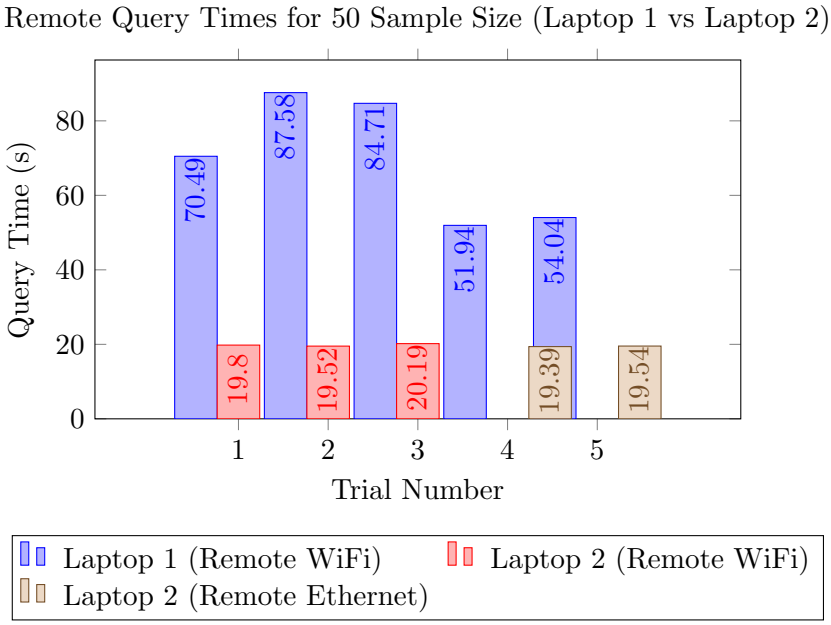


Figure 5.1: Comparison of Remote Query Times for 50 Sample Size (Laptop 1 vs Laptop 2).

Impact of Network Conditions

The critical role of network conditions is evident when comparing the results across home Wi-Fi, home Ethernet, campus Wi-Fi, and campus Ethernet, as summarized in Table 4.2 from Chapter 4.2.1. Key findings include:

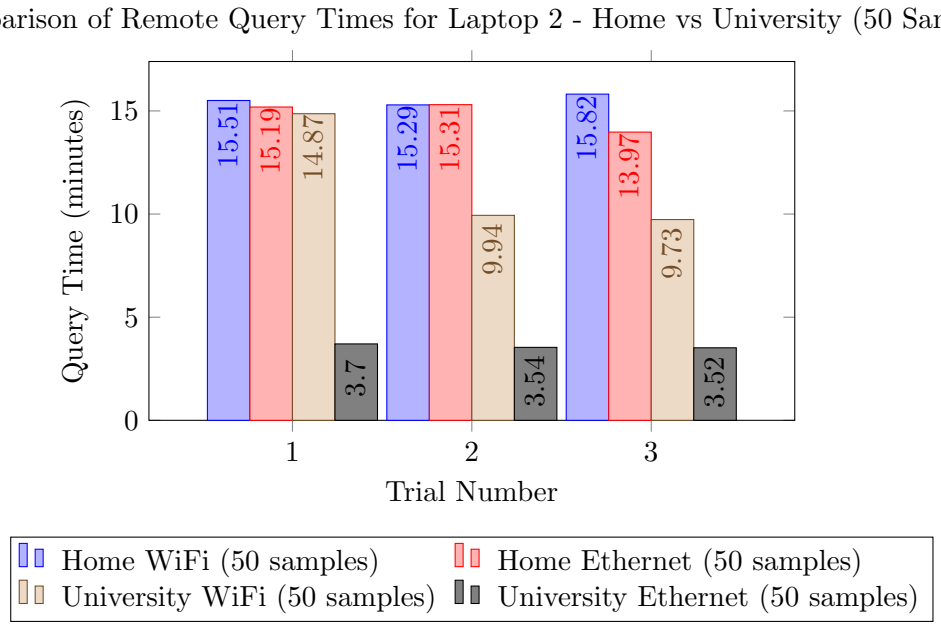


Figure 5.2: Comparison of Remote Query Times for Laptop 2 - Home vs University (50 Samples).

- **Bandwidth and Latency:**
  Slower upload/download speeds and higher latency on home Wi-Fi led to significantly higher query times compared to campus connections. For example, on home Wi-Fi, average query times for 50-query samples ranged from 19.5 to 20.2 seconds, while campus cable reduced this to just 4.5 seconds per query.

- **Consistency of Results:**
  High-speed connections, such as campus cable, not only reduced average query times but also minimized variability, ensuring more predictable performance.

Local CIFF storage consistently delivered the fastest query times with minimal variability, largely due to the absence of network dependencies. In contrast, remote storage, while requiring robust network infrastructure to match local performance, demonstrated competitive results on high-speed connections like campus cable. This highlights the trade-offs between the two approaches: remote storage offers significant scalability and accessibility advantages but demands reliable, low-latency networks to minimize overhead and achieve optimal performance.

These findings emphasize that while local storage remains superior for

query execution in isolated environments, remote storage is a viable alternative when supported by high-speed networks. The next section examines the breakdown of query execution times, including the proportion spent on specific operations such as reading Parquet files.

5.2.2 Read_Parquet Time vs Overall Query Time Analysis

The comparison of "READ_PARQUET" time and total query time is critical for understanding where the majority of processing time is spent during remote queries. The total query time reported in this section is derived directly from DuckDB's `EXPLAIN ANALYZE` outputs, representing the subtotal of all operator times within the query execution pipeline. This approach provides an accurate measurement of the actual time spent processing the query itself, free from the external overhead introduced by the Python environment or other external factors.

It is important to distinguish this metric from the wall time discussed in the previous chapter. Wall time encompasses the full duration from the initiation of the Python script to its completion, including additional operations like setting up the query, fetching results, and any processing outside DuckDB. By isolating the operator times using `EXPLAIN ANALYZE`, the total query time reflects the efficiency of DuckDB's execution engine, offering a more granular view of performance specific to database processing.

A significant portion of the query execution time in the remote setup is spent on reading Parquet files, making the "READ_PARQUET" operator a key focus for understanding performance bottlenecks. By analyzing the proportion of total query time consumed by the read operation, we can better assess how network conditions and storage configurations influence overall efficiency.

Table 5.2 highlights the relationship between "READ_PARQUET" times and total query times across various devices and network conditions. These observations reveal that, while the read operation constitutes the majority of the query time, other contributing factors, such as CPU-bound processing and thread management, can still impact the total execution time.

Device/Network Condition	Sample Size	Total Time (s)	Read Parquet Time (s)	Difference (s)
Laptop 1 (Home WiFi)	10 queries	41.1004	38.2716	2.8288
Laptop 1 (Home WiFi)	10 queries	39.2591	36.2114	3.0477
Laptop 2 (Home WiFi)	50 queries	18.3366	18.1091	0.2275
Laptop 2 (Home WiFi)	50 queries	18.0715	17.8624	0.2091
Laptop 2 (Home Ethernet)	50 queries	17.6008	17.3704	0.2304
Laptop 2 (Home Ethernet)	50 queries	16.0841	15.7927	0.2914
Laptop 2 (University WiFi)	50 queries	10.3110	10.1173	0.1937
Laptop 2 (University WiFi)	50 queries	17.0860	16.7321	0.3539
Laptop 2 (University WiFi)	50 queries	16.9374	16.6274	0.3100
Laptop 2 (University Ethernet)	50 queries	2.1693	1.9586	0.2107
Laptop 2 (University Ethernet)	50 queries	2.1760	1.9797	0.1963
Laptop 2 (University Ethernet)	50 queries	2.1731	1.9764	0.1967

Table 5.2: Comparison of Read Parquet vs. Overall Times Across Devices and Networks. Differences are calculated as Total Time - Read Parquet Time.

The results show that "READ_PARQUET" operations account for a large percentage of total query time, particularly in slower network environments. For instance, on home Wi-Fi, the read operation (strictly comparing Laptop 2 results) constituted over 98% of the total query time for most trials, whereas on faster connections, such as the university's cable network, this proportion dropped to approximately 90%. This variation is attributed to differences in network latency and bandwidth, as faster networks reduce the time spent transferring data.

Interestingly, while the difference between "READ_PARQUET" and total query times is generally small, it is not negligible. On local setups, this gap is more pronounced, with CPU-bound processing contributing more significantly to the total time due to the absence of network transfer delays. For remote setups, however, the processing overhead becomes almost imperceptible, as the data transfer dominates.

These findings suggest that optimizing the "READ_PARQUET" operator, for example, by reducing partition sizes to better align with average query sizes, could yield meaningful performance improvements. Such adjustments would limit the amount of redundant data transferred during query execution, thereby reducing the overall time required.

Figure 5.3 and Figure 5.4 provide visual comparisons of "READ_PARQUET" and total query times under home and university network conditions, emphasizing the impact of faster connections on narrowing the time differential.

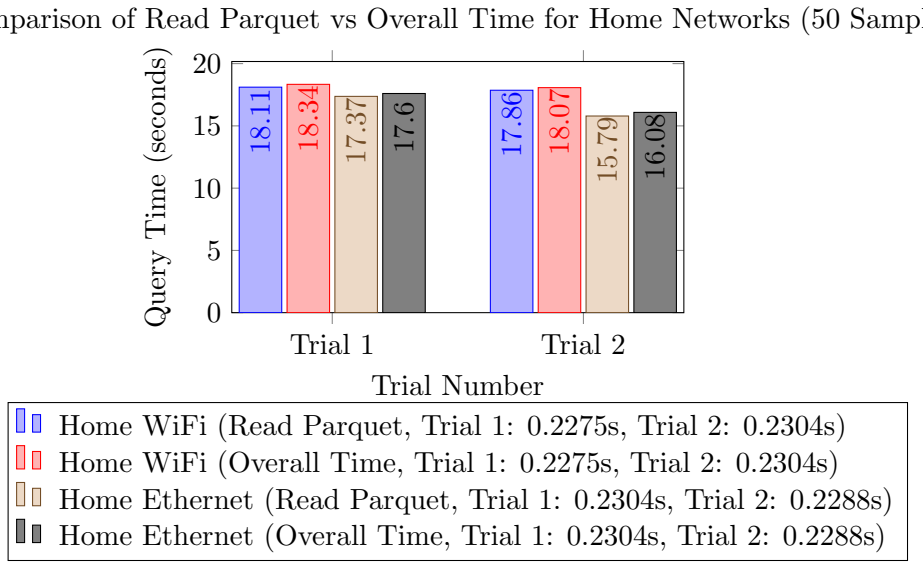


Figure 5.3: Comparison of Read Parquet vs Overall Time for Home Networks (50 Samples).

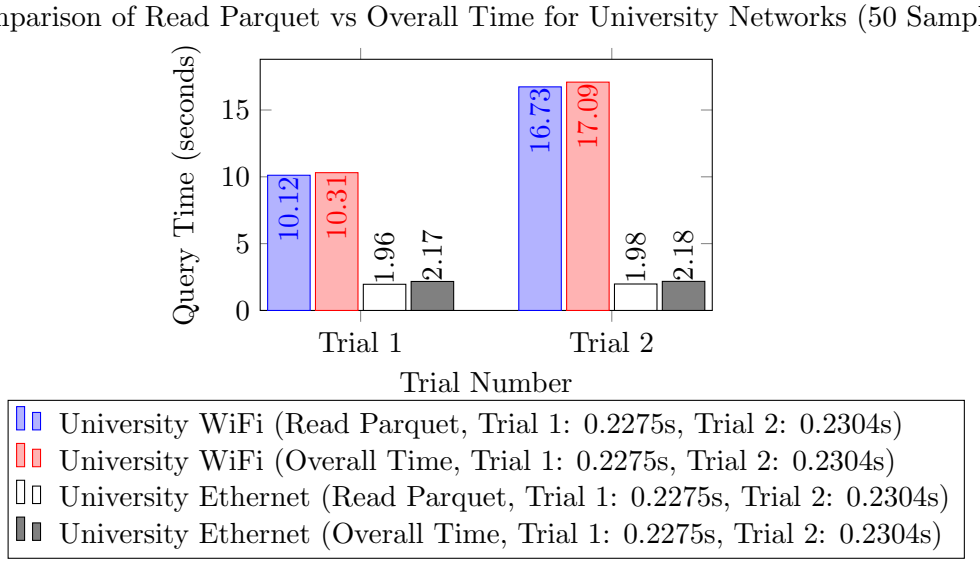


Figure 5.4: Comparison of Read Parquet vs Overall Time for University Networks (50 Samples).

In summary, while "READ_PARQUET" remains the primary contributor to query times in remote setups, its proportionate impact is mitigated

by faster network connections. This underscores the importance of considering network conditions and partitioning strategies when deploying remote storage solutions for high-frequency query workloads.

Chapter 6

Analysis and discussion

This chapter synthesizes the findings from the results presented in Chapter 4 and connects them to the research objectives outlined in Chapter 3. The primary aim is to interpret the observed differences between locally stored CIFF files and remotely stored Parquet files in terms of storage efficiency, query performance, and practical applicability.

Through a detailed analysis, this chapter highlights the trade-offs inherent in each storage strategy, evaluates the influence of external factors such as network conditions and hardware, and provides insights into the broader implications for CIFF-based systems in information retrieval. Limitations of the research are acknowledged, and directions for future exploration are suggested.

6.1 Key Findings

The experimental results revealed several key insights into the comparative performance of local CIFF storage and remote Parquet storage. These findings highlight trade-offs inherent in each storage strategy and provide a nuanced understanding of their implications.

Storage Efficiency

Local CIFF storage required only 2.9 GB after deduplication and compaction, compared to the remote Parquet setup's 6.2 GB. This difference stems from the compact nature of CIFF files and DuckDB's default Snappy compression. However, the scalability of remote storage makes it better suited for larger datasets.

Query Performance

Local storage consistently outperformed remote storage across all tests. The analysis of operator timings using DuckDB's `EXPLAIN ANALYZE` revealed that the "READ_PARQUET" operator accounted for 90% or more of the query time in remote setups. This highlights the critical role of network bandwidth and latency in determining remote query performance.

Impact of Hardware and Network Conditions

Tests conducted on Laptop 2 revealed that robust network infrastructure, such as campus cable, allowed remote query times to approach those of local storage. However, older hardware, such as Laptop 1, exacerbated performance differences due to outdated network cards and slower components for handling data transfers.

Practical Applicability

The results demonstrated that remote storage, while less performant, offers scalability and accessibility advantages for distributed systems. These findings provide actionable insights for designing storage systems tailored to specific use cases.

6.2 Trade-Offs in Storage Efficiency vs. Query Performance

The trade-off between storage efficiency and query performance is central to the choice between local and remote storage.

1. **Local Storage: Optimal Query Performance:**
   Local CIFF storage offers unmatched query performance due to the absence of network dependencies. Its compact nature makes it suitable for small to medium-sized datasets where scalability is not a concern. However, local storage lacks the accessibility and flexibility required for distributed systems.

2. **Remote Storage: Scalability with Limitations:**
   Remote Parquet storage offers scalability and accessibility, critical for large-scale and distributed systems. However, its performance is heavily influenced by network conditions, with significant delays observed in environments with high latency. Partitioning overhead further compounds these delays for frequent or complex queries.

3. **Balancing the Trade-Offs:**
   The choice between local and remote storage depends on the application's specific needs. For small, real-time systems, local storage is

ideal, while remote storage is better suited for scalable, distributed setups with robust network infrastructure.

6.3 Influence of Network Conditions and Hardware

The performance of remote storage solutions in this study was found to be heavily influenced by two key factors: network conditions and the hardware configurations of the testing devices. While local storage exhibited consistently superior performance due to its lack of network dependency, the variability in remote query performance highlighted the importance of these external influences.

Network Conditions

The quality of the network connection played a critical role in remote query performance. Queries executed over slower connections, such as home Wi-Fi, experienced significantly higher latency compared to those performed on faster connections, such as university Ethernet. For instance, the average query time for 50 samples on laptop 2 using home Wi-Fi was 19.8 seconds, whereas the same workload on a university cable connection reduced the query time to approximately 4.5 seconds. Similarly, 100-sample queries showed significant reductions in query times when tested over high-speed, low-latency networks like the university's Ethernet connection.

These results underscore that the viability of remote storage solutions is highly dependent on the availability of reliable, low-latency network infrastructure. While remote setups offer scalability and accessibility, they come with the inherent trade-off of increased sensitivity to network conditions, which may lead to performance bottlenecks in less-optimized environments.

Hardware Influence

The hardware specifications of the testing devices also influenced query performance. Both laptops used in this study featured SSDs and 16 GB of RAM, but their CPU configurations varied. Laptop 1, with an Intel Core i5-8300H (4 cores, 8 threads), generally lagged behind laptop 2, equipped with an AMD Ryzen 7 4800H (8 cores, 16 threads). However, for fairness, laptop 2 was restricted to using only 8 threads, ensuring a comparable testing environment.

Despite these controlled conditions, the performance differences between the two devices were noticeable in the remote query tests, particularly on

high-speed connections. The faster CPU and better multithreading capabilities of laptop 2 allowed it to handle the network overhead of remote queries more efficiently. This demonstrates that while network conditions play a dominant role in remote performance, hardware configurations can amplify or mitigate these effects, depending on the workload.

In addition to CPU and RAM, the network-related hardware of the devices also played a role in query performance. The table comparing read parquet times versus overall times reveals that Laptop 1 took longer to execute queries compared to Laptop 2, even under identical network conditions. For instance, for 50-query samples, the read parquet times on Laptop 1 were significantly higher than those on Laptop 2. This discrepancy may be attributed to the age of the devices: Laptop 1, being around 8 years old, likely features a less advanced network card and slower internal components for processing network data. Laptop 2, as a newer model, benefits from more modern network hardware, which can process network-related tasks more efficiently. This hardware difference highlights the impact of not just network quality but also the capability of the device to handle network traffic and data transfers effectively.

By combining insights from CPU, RAM, and network hardware, the findings suggest that the efficiency of remote storage setups is influenced by a holistic combination of network infrastructure and device capabilities. While faster networks can mitigate some performance issues, suboptimal hardware may still act as a bottleneck, emphasizing the need for balance between hardware and network resources.

6.4 Limitations and Influence of Data Imperfections

This section addresses limitations of the study and the role of data imperfections, such as zero-length documents and skipped queries. Key recommendations include improved preprocessing and the exploration of more sophisticated partitioning strategies.

Limitations of the Study Design

1. **Testing Environments and Variability:**
   The experiments were conducted across different devices and network setups, which, while reflective of real-world conditions, introduced variability. For example, differences in laptop hardware (including the network cards) impacted the query times, as highlighted by discrepancies in the read parquet times between Laptop 1 and Laptop 2.

Although Laptop 2 was restricted to 8 threads to match Laptop 1, its newer network hardware and CPU likely contributed to faster remote query execution, suggesting that some results may not generalize to environments with older hardware.

2. **Network Conditions:**
   The dependency on external factors, such as network speed and stability, was a notable limitation for remote queries. Variability in bandwidth and latency, particularly for non-cable connections, influenced the performance of the remote setup. While attempts were made to standardize testing conditions, the unavoidable differences in network quality likely introduced additional noise into the data.

3. **Limited Query Sample Sizes:**
   Although multiple query sample sizes (e.g., 50 and 100 queries) were tested, the scope of the study was constrained by time and computational resources. Larger sample sizes or a broader variety of query types might provide a more comprehensive understanding of performance across storage systems.

4. **Simplification of Partitioning Strategy:**
   The partitioning of the remote Parquet files was static and not optimized for the query patterns observed during testing. A dynamic approach tailored to average query lengths or data access patterns could reveal additional performance gains, but such an implementation was beyond the scope of this project.

Influence of Data Imperfections

1. **Initial Dataset Flaws:**
   The original dataset contained non-unique document IDs, which were corrected during data preparation. This issue was a result of improper data handling in the initial CIFF representation, and its resolution ensured that document representations were accurate. However, this correction process introduced preprocessing overhead, which could be avoided in future studies with cleaner datasets.

2. **Zero-Length Documents:**
   Approximately 3.9% of the dataset consisted of documents with zero-length content. These entries, while retained to simulate real-world scenarios, likely had minimal influence on query performance but contributed to storage inefficiency. Their inclusion provides a realistic view of how such imperfections might impact results but does not allow for comparisons under ideal data conditions.

3. **Duplicate and Redundant Postings:**
   While duplicates were removed to ensure consistent results, the initial presence of these redundant postings could have influenced early iterations of the experiments. Their removal improved storage efficiency and likely reduced term frequency inflation, enabling more accurate rankings with BM25 [28].

4. **Skipped Queries:**
   Queries containing terms not present in the dataset were skipped during the experiments. While this was necessary to prevent errors in execution, it resulted in slightly reduced sample sizes for average query time calculations. Future work could focus on including fallback mechanisms for such queries to better simulate real-world search scenarios.

Implications of Data Imperfections

The study demonstrates that addressing data imperfections is crucial for obtaining valid and replicable results. While this project resolved critical issues such as non-unique document IDs and redundant postings, other imperfections like zero-length documents remained unaddressed, reflecting a trade-off between data realism and idealized conditions. Future research should consider further preprocessing to eliminate such imperfections and explore their influence on both storage efficiency and query performance.

By acknowledging these limitations and data imperfections, this study provides a transparent foundation for future work in the field, ensuring that its findings can be interpreted within the context of these constraints.

6.5 Practical Implications and recommendations

The results of this study provide several practical implications for the design and implementation of storage systems in information retrieval applications, especially for use with CIFF files. By analyzing both local and remote storage systems, this research highlights key considerations that can inform decision-making for diverse use cases.

The findings demonstrate that local storage offers significant advantages in terms of query performance, particularly for setups with limited scalability requirements. For example, the low latency and high-speed access observed with locally stored CIFF files make it an ideal choice for smaller-scale environments where hardware resources can be directly allocated to the storage system. However, this approach is constrained by the physical limitations of local hardware, such as storage capacity and scalability.

On the other hand, remote storage, such as the Minio S3 bucket used in this research, provides a viable alternative in scenarios requiring distributed access and scalability. While the performance of remote queries is highly dependent on network conditions, the experiments conducted under high-speed, low-latency conditions (e.g., university Ethernet) show that remote storage can achieve competitive results. The inherent trade-offs between accessibility and performance underline the importance of tailoring the storage setup to the specific requirements of the application.

Recommendations

To maximize the potential of both local and remote storage systems, the following recommendations are proposed:

1. **Optimizing for Use Case:**
    - For applications with small-scale datasets and low latency requirements, local storage is recommended due to its consistent performance and reduced network dependencies.
    - For large-scale systems or distributed teams, remote storage provides scalability and accessibility benefits but requires careful consideration of network infrastructure to minimize latency and bandwidth limitations.
2. **Improving Network Performance:**
   Given the significant influence of network conditions on remote query performance, practitioners should prioritize high-speed, low-latency connections when deploying remote storage systems. For instance, using wired connections over wireless can yield substantial (more than linear!) performance gains, as demonstrated by the query times for university Ethernet versus Wi-Fi in this study.
3. **Handling Data Imperfections:**
   Data quality plays a critical role in storage efficiency and query performance. Future implementations should incorporate preprocessing steps to handle zero-length documents and duplicate data more comprehensively. Ensuring clean and consistent datasets will improve replicability and the validity of performance evaluations.

Partitioning Recommendations

One of the key findings in this research is the influence of partitioning on query performance in remote storage. The static partitioning strategy used in this study, while effective, does not fully optimize the performance for specific query patterns or network conditions. Future work should explore semi-dynamic and dynamic partitioning methods, where partition sizes are

determined based on average query lengths and available network bandwidth.

For instance, by analyzing the average data transfer requirements per query and aligning them with the network's upload and download speeds, it may be possible to minimize the time spent reading Parquet files while maximizing throughput. This approach would require a feedback loop to adjust partition sizes dynamically, ensuring that the storage system adapts to varying workloads and infrastructure conditions, which may prove to be more difficult than its worth.

Final Recommendations

Overall, this research highlights the importance of balancing storage efficiency, query performance, and scalability when selecting a storage system for using CIFF files. By considering the specific demands of the application, such as dataset size, access patterns, and network conditions, practitioners can make informed decisions to optimize their systems. Future studies should continue to investigate hybrid approaches, such as caching frequently accessed partitions locally or combining local and remote storage to achieve the best of both worlds.

6.6 Future Research Directions

This study has laid the groundwork for understanding the storage advantages and drawbacks, and performance trade-offs between local and remote storage systems for CIFF files. However, several avenues remain open for further exploration, offering the potential to enhance the efficiency, scalability, and applicability of these systems in diverse contexts.

Refinement of Current Methods

Future research should investigate strategies to optimize current methodologies. One key direction is the refinement of partitioning strategies for remote storage. The static partitioning approach used in this study could be replaced with dynamic partitioning techniques, where partition sizes are tailored to match average query lengths and network speeds. This adaptation could significantly improve query performance by reducing data transfer overhead during queries.

Alternative compression techniques also warrant exploration. While Snappy compression was effective in this study, other methods, such as Zstandard or Brotli, might offer better storage efficiency without compromising query speed. Comparative analysis of these techniques could provide valuable insights into their applicability for inverted index storage.

Future research could also focus on incorporating near-duplicate detection within these settings to evaluate its potential impact on query speed. Additionally, exploring datasets in different languages or domain-specific content could help create a more scenario-specific environment or broaden the scope of the data. Such expansions may also reveal alternative approaches to the ranking model employed in this study.

Finally, addressing data imperfections remains a critical area for improvement. Methods for detecting and handling near-duplicates, as well as resolving zero-length documents, could enhance dataset quality, improving both storage efficiency and query accuracy.

Exploration of New Approaches

Expanding beyond the current scope, future research could explore hybrid storage systems that combine the strengths of local and remote storage. For instance, frequently accessed data could be cached locally while leveraging remote storage for scalability. This hybrid approach might mitigate the limitations of each storage type.

Emerging file formats such as ORC and Avro represent another area of interest. Although these formats were not utilized in this study due to compatibility limitations with DuckDB, future investigations could evaluate their potential for storing inverted index files. Additionally, integrating these formats into query engines like DuckDB could open up new possibilities for optimizing storage and query performance.

Caching strategies for remote storage also deserve further study. By analyzing access patterns and preloading commonly queried partitions, it might be possible to reduce query latency while maintaining the scalability of remote setups.

CIFF as a DuckDB database file in S3

As a last-minute experimental setup, the 2.8 GB compacted `.db` file was uploaded to the S3 storage system. A modified version of the code, originally designed for local querying, was adapted to attach and query the database file remotely. The file occupied approximately the same storage space on the remote setup as on the local setup, but achieved significantly faster query times than Parquet, with a 100-query sample completing in 66 seconds over home Wi-Fi using Laptop 2. This represents an approximately tenfold improvement in query efficiency compared to the results obtained using Ethernet on the Parquet dataset. Possible explanations for the additional cost include that 1) deserialization of Parquet's internal formats is much more expensive than querying the DuckDB data directly and 2) not

all query optimization techniques that are used over data represented in the DuckDB format are available when querying data stored as Parquet. Future research could further investigate this promising approach, examining its potential for optimizing remote query performance and exploring its broader applicability.

Broader Contexts and Applications

The findings of this study could be applied to larger datasets or real-world production systems to validate their scalability and robustness. Such experiments would provide valuable insights into the practical implications of using CIFF and Parquet in different operational contexts.

Comparative studies involving alternative query engines could also shed light on how engine-specific optimizations influence the performance of CIFF loaded into database files and Parquet files. Engines such as Spark SQL or PrestoDB could be tested for compatibility and performance with these formats.

Finally, the interplay between hardware advancements and network conditions should be explored further. For example, as network hardware evolves, the performance gap between local and remote storage may narrow, necessitating ongoing evaluation of their trade-offs.

Conclusion

Future research should focus on building upon the insights gained in this study to develop more robust, scalable, and efficient storage systems. By addressing the outlined directions, researchers can advance the field of information retrieval, paving the way for innovative solutions to meet the growing demands of data-driven applications.

Chapter 7

Conclusions

This research presents a comparative evaluation of CIFF-based inverted index storage in local and remote environments, focusing on storage efficiency, query performance, and scalability. By leveraging DuckDB for querying and Minio S3 for remote storage, this study highlights the potential and limitations of CIFF files as a standardized solution for inverted indexing. The conclusions provide actionable insights into their applications and implications for information retrieval systems.

7.1 Summary of Findings

This study revealed several critical insights into CIFF-based storage systems. Local CIFF storage loaded into DuckDB demonstrated remarkable storage efficiency, requiring only 2.9 GB post-compaction, compared to the remote Parquet setup's 6.2 GB. This compactness makes it highly suitable for small to medium datasets. Query performance consistently favored local storage, which achieved minimal latency due to the absence of network dependencies. In contrast, remote setups were heavily influenced by network conditions, with the "READ_PARQUET" operator accounting for the majority of query execution time. These findings highlight a trade-off: while local storage excels in speed and efficiency, remote storage offers scalability and centralized access, making it suitable for distributed systems. Additionally, the study underscored the significant impact of hardware and network infrastructure on remote performance. Advanced hardware and robust network connections were key factors in narrowing the performance gap between local and remote setups.

7.2 Limitations and Future Directions

While this research provides valuable insights, several limitations must be acknowledged. Residual imperfections in the dataset, such as skipped queries

and unoptimized partitioning, may have introduced minor biases in the results, highlighting the need for cleaner datasets in future work. Additionally, static partitioning strategies, while effective, were not tailored to specific query patterns, limiting potential efficiency gains. Future research should explore dynamic partitioning methods that adapt to query characteristics and network performance. Furthermore, the study emphasized the importance of network and hardware variability, as advanced devices and robust connections significantly impacted remote query times. Expanding this research to incorporate a broader range of hardware and network configurations would provide a more comprehensive understanding of CIFF's scalability and performance in local and remote contexts. Future investigations should also consider hybrid storage models that integrate the advantages of local and remote setups, as well as CIFF's adaptability to larger datasets and diverse retrieval tasks.

7.3 Final Thoughts

This study underscores the relevance and versatility of CIFF files as a standardized solution for inverted index storage in both local and remote environments. By exploring the trade-offs between storage efficiency, query performance, and scalability, this research provides actionable guidance for optimizing CIFF-based systems.

The findings also emphasize the importance of infrastructure alignment: local setups thrive in latency-sensitive scenarios, while remote storage unlocks potential for distributed and scalable applications. With continued research into partitioning strategies, hybrid approaches, and network optimization, CIFF files can become an integral component of next-generation search engine architectures.

Ultimately, this work serves as a foundation for advancing the state of the art in information retrieval, enabling researchers and practitioners to harness the full potential of CIFF-based systems in diverse, real-world contexts.

Bibliography

[1] Apache Software Foundation. Orc specification: Orcv1. https://orc.apache.org/specification/ORCv1/, 2025. Accessed: 2025-01-05.

[2] Bonetti, L., and Torroni, P. *Design and Implementation of a Realworld Search Engine Based on Okapi Bm25 and Sentencebert*. PhD thesis, MS Thesis, Department of Computer Science and Engineering, Alma Mater , 2021.

[3] Christopher, D. M., Prabhakar, R., and Hinrich, S. Introduction to information retrieval, 2008.

[4] Craswell, N., Campos, D., Mitra, B., Yilmaz, E., and Billerbeck, B. Orcas: 18 million clicked query-document pairs for analyzing search. *arXiv preprint arXiv:2006.05324* (2020).

[5] de Vries, A. Ciff2duckdb: Load ciff indices into tables in duckdb. https://github.com/arjenpdevries/CIFF2DuckDB, 2024. Accessed: 2024-10-18.

[6] DuckDB Development Team. File formats - duckdb. https://duckdb.org/docs/guides/performance/file_formats.html. Accessed: 2024-10-10.

[7] DuckDB Development Team. Full-text search extension. https://duckdb.org/docs/extensions/full_text_search.html#match_bm25-function, 2024. Accessed: 2024-10-10.

[8] DuckDB Development Team. Reclaiming space. https://duckdb.org/docs/operations_manual/footprint_of_duckdb/reclaiming_space.html, 2024. Accessed: 2024-12-17.

[9] DuckDB Development Team. Duckdb documentation: Constant encoding. https://duckdb.org/2022/10/28/lightweight-compression.html#constant-encoding, 2025. Accessed: 2025-01-11.

[10] DuckDB Development Team. Duckdb documentation: Profiling and execution visualization. `https://duckdb.org/docs/dev/profiling.html`, 2025. Accessed: 2025-01-05.

[11] DuckDB Development Team. Duckdb documentation: Reading and writing parquet files. `https://duckdb.org/docs/data/parquet/overview.html`, 2025. Accessed: 2025-01-05.

[12] DuckDB Team. Discussion on GitHub: 6529. `https://github.com/duckdb/duckdb/discussions/6529`, 2024. Accessed: 2024-12-17.

[13] Durner, D. *High-Performance Query Processing in the Cloud*. PhD thesis, Technische Universität München, 2024.

[14] Engine, P. Common index file format (ciff). `https://github.com/pisa-engine/ciff`, 2023. Accessed: 2024-09-15.

[15] Gaffney, K. P., Prammer, M., Brasfield, L., Hipp, D. R., Kennedy, D., and Patel, J. M. Sqlite: past, present, and future. *Proc. VLDB Endow. 15*, 12 (Aug. 2022), 3535–3547.

[16] Gharaibeh, A. *Effective Use of SSDs in Database Systems*. PhD thesis, University of Waterloo, 2013. Accessed: 2025-01-05.

[17] Hu, J., Wang, Y., Shi, F., and Xu, C. Compared analysis of row-based storage and column-based storage. In *2018 Eighth International Conference on Instrumentation Measurement, Computer, Communication and Control (IMCCC)* (2018), pp. 168–173.

[18] Jaranilla, C., and Choi, J. Requirements and trade-offs of compression techniques in key–value stores: A survey. *Electronics 12*, 20 (2023).

[19] Li, Z., Zhang, Y., Liu, Y., Xu, T., Zhai, E., Liu, Y., Ma, X., and Li, Z. A quantitative and comparative study of network-level efficiency for cloud storage services. *ACM Trans. Model. Perform. Eval. Comput. Syst. 4*, 1 (Jan. 2019).

[20] Lim, K.-T., Pilman, M., Kossmann, D., and Kraska, T. Choosing a cloud dbms: Architectures and tradeoffs. In *Proceedings of the VLDB Endowment (PVLDB)* (2014), vol. 7, VLDB Endowment, pp. 1102–1113.

[21] Lin, J., Mackenzie, J., Kamphuis, C., Macdonald, C., Mallia, A., Siedlaczek, M., Trotman, A., and de Vries, A. Supporting interoperability between open-source search engines with the common index file format. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval* (2020), ACM, pp. 2149–2152.

[22] Liu, P.-J., Li, C.-P., and Chen, H. Enhancing storage efficiency and performance: A survey of data partitioning techniques. *Journal of Computer Science and Technology 39*, 2 (2024), 346–368.

[23] Mahapatra, A. K., and Biswas, S. Inverted indexes: Types and techniques. *International Journal of Computer Science Issues (IJCSI) 8*, 4 (2011), 384.

[24] Microsoft. ORCAS: Open Resource for Click Analysis in Search. `https://microsoft.github.io/msmarco/ORCAS`, 2020. Accessed: 2024-10-10.

[25] Nambiar, A., and Mundra, D. An overview of data warehouse and data lake in modern enterprise data management. *Big Data and Cognitive Computing 6*, 4 (2022).

[26] OpenWebSearch.EU. OWLer - OpenWebSearch Crawler. `https://openwebsearch.eu/owler`, 2024. Accessed: 2024-10-10.

[27] OSIRRC. Common index file format (ciff). `https://github.com/osirrc/ciff/`, 2023. Accessed: 2024-09-15.

[28] Phuc, N. M. Using bm25 weighting and cluster shrinkage for detecting duplicate bug reports. *International Journal of Advanced Research in Computer and Communication Engineering 7*, 11 (2018), 71–77.

[29] Raasveldt, M., and Mühleisen, H. Duckdb: an embeddable analytical database. SIGMOD '19, Association for Computing Machinery, p. 1981–1984.

[30] Schuth, A., Sietsma, F., Whiteson, S., and Rijke, M. Optimizing base rankers using clicks: A case study using bm25. vol. 8416.

[31] Silberschatz, A., Korth, H. F., and Sudarshan, S. *Database System Concepts*, 7th ed. McGraw-Hill Education, 2019. Covers data storage structures and indexing related to storage efficiency.

[32] Sweeney, A., et al. Scalability in the xfs file system. In *Proceedings of the USENIX 1996 Annual Technical Conference* (1996), USENIX Association.

[33] Vohra, D. *Apache Parquet*. Apress, Berkeley, CA, 2016, pp. 325–335.

[34] Zobel, J., Moffat, A., and Ramamohanarao, K. Inverted files versus signature files for text indexing. *ACM Transactions on Database Systems (TODS) 23*, 4 (1998), 453–490.

Appendix A

Appendix

BM25 snippet: (discussed in chapter 4.3.3)

```
"""
    ...
    p.docid,
    LOG((SELECT num_docs FROM stats) / (1 + q.df)) *
        (p.tf * (1.5 + 1)) /   (p.tf + 1.5 * (1 -
       0.75 + 0.75 * (d.len / (SELECT avgdl FROM
       stats))))
    AS bm25termscore
    ...
"""
```

Remote query snippet:

```
def run_queries(queries):
    for query in queries:

        # Build the queryterms CTE
        queryterms_cte = f"""
        WITH queryterms AS (
            SELECT term, termid, df, crange
            FROM dict
            WHERE term IN ({", ".join(f"'{term}'"
               for term in query)})
        )
        """

        # Extract termids of query terms in current
           query into a Python list
        termids_result = con.execute(f"""
        SELECT termid FROM dict
```

```
        WHERE term IN ({", ".join(f"'{term}'" for term in query)});
        """).fetchall()

        # Query terms without an id will return nothing in the lines above
        # In the case an entire query returns nothing (eg. single term that doesn't have id), that is dealt with here
        if not termids_result:
            print(f"No terms found in dictionary for query: {query}")
            continue

        # Isolate the termids from the result
        termids = [row[0] for row in termids_result]

        # Construct the termscores CTE
        termscores_cte = " UNION ".join([
            f"""SELECT
                    p.docid,
                    LOG((SELECT num_docs FROM stats) / (1 + q.df)) *
                        (p.tf * (1.5 + 1)) /
                        (p.tf + 1.5 * (1 - 0.75 + 0.75 * (d.len / (SELECT avgdl FROM stats)))
                    ) AS bm25termscore
                FROM postings p, queryterms q, docs d
                WHERE (p.termid = {termid}
                    AND p.thash = MOD(hash({termid}), 40)
                    AND p.termid = q.termid
                    AND p.docid = d.docid)"""
            for termid in termids
        ])

        # Combine everything into the final SQL query
        scoreBM25 = f"""
        {queryterms_cte},
        termscores AS (
            {termscores_cte}
```

```
        ),
        docscores AS (
            SELECT docid, SUM(bm25termscore) AS
               bm25score
            FROM termscores
            GROUP BY docid
        )
        SELECT ds.docid, ds.bm25score, di.name
        FROM docscores ds
        JOIN docs di ON ds.docid = di.docid
        ORDER BY ds.bm25score DESC;
        """

        result = con.execute(scoreBM25).fetchall()
```

Local query snippet:

```
def run_queries(queries):
    for query in queries:

        # Build the queryterms CTE
        queryterms_cte = f"""
        WITH queryterms AS (
            SELECT term, termid, df
            FROM dict
            WHERE term IN ({", ".join(f"'{term}'"
               for term in query)})
        )
        """

        # Extract termids of query terms in current
           query into a Python list
        termids_result = con.execute(f"""
        SELECT termid FROM dict
        WHERE term IN ({", ".join(f"'{term}'" for
           term in query)});
        """).fetchall()

        # Query terms without an id will return
           nothing in the lines above
        # In the case an entire query returns
           nothing (eg. single term that doesn't
           have id), that is dealt with here
        if not termids_result:
```

```
            print(f"No terms found in dictionary for query: {query}")
            continue

        # Isolate the termids from the result
        termids = [row[0] for row in termids_result]

        # Construct the termscores CTE
        termscores_cte = " UNION ".join([
            f"""SELECT
                    p.docid,
                    LOG((SELECT num_docs FROM stats) / (1 + q.df)) *
                        (p.tf * (1.5 + 1)) /
                        (p.tf + 1.5 * (1 - 0.75 + 0.75 * (d.len / (SELECT avgdl FROM stats)))
                    ) AS bm25termscore
                FROM postings p, queryterms q, docs d
                WHERE (p.termid = {termid}
                    AND p.thash = MOD(hash({termid}), 40)
                    AND p.termid = q.termid
                    AND p.docid = d.docid)"""
            for termid in termids
        ])

        # Combine everything into the final SQL query
        scoreBM25 = f"""
        {queryterms_cte},
        termscores AS (
            {termscores_cte}
        ),
        docscores AS (
            SELECT docid, SUM(bm25termscore) AS bm25score
            FROM termscores
            GROUP BY docid
        )
        SELECT ds.docid, ds.bm25score, di.name
        FROM docscores ds
        JOIN docs di ON ds.docid = di.docid
```

```
58          ORDER BY ds.bm25score DESC;
59          """
60
61          result = con.execute(scoreBM25).fetchall()
```

Device/Network Condition	Connection Condition	Sample Size	Total Wall Time (s)	Average Time (s)
Laptop 1 (Local)				
Laptop 1 (Local)	Local	100 queries	346.9666 secs	3.6142 secs
Laptop 1 (Local)	Local	100 queries	330.1122 secs	3.5118 secs
Laptop 1 (Local)	Local	100 queries	344.4885 secs	3.6648 secs
Laptop 1 (Local)	Local	50 queries	201.1285 secs	4.2793 secs
Laptop 1 (Local)	Local	50 queries	193.9682 secs	4.1270 secs
Laptop 1 (Local)	Local	50 queries	192.6363 secs	4.0986 secs
Laptop 1 (Remote)				
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	100 queries	82.8770 min	52.9002 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	100 queries	129.8060 min	82.8549 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	100 queries	116.8501 min	74.5852 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	100 queries	102.4494 min	65.3932 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	50 queries	55.2170 min	70.4898 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	50 queries	68.6042 min	87.5798 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	50 queries	66.3532 min	84.7062 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	50 queries	40.6886 min	51.9429 secs
Laptop 1 (Remote)	Home Wi-Fi (Science VPN)	50 queries	42.3340 min	54.0435 secs
Laptop 2				
Laptop 2 (Remote)	Home Wi-Fi (EduVPN)	50 queries	15.5065 min	19.7955 secs
Laptop 2 (Remote)	Home Wi-Fi (EduVPN)	50 queries	15.2927 min	19.5226 secs
Laptop 2 (Remote)	Home Wi-Fi (EduVPN)	50 queries	15.8150 min	20.1894 secs
Laptop 2 (Remote)	Home Wi-Fi (EduVPN)	100 queries	29.6216 min	18.9074 secs
Laptop 2 (Remote)	Home Wi-Fi (EduVPN)	100 queries	29.2819 min	18.6906 secs
Laptop 2 (Remote)	Home Wi-Fi (EduVPN)	100 queries	29.9220 min	19.0991 secs
Laptop 2 (Remote)	Home Ethernet (EduVPN)	50 queries	15.1899 min	19.3914 secs
Laptop 2 (Remote)	Home Ethernet (EduVPN)	50 queries	15.3051 min	19.5384 secs
Laptop 2 (Remote)	Home Ethernet (EduVPN)	50 queries	13.9741 min	17.8393 secs
Laptop 2 (Remote)	Home Ethernet (EduVPN)	100 queries	30.4696 min	19.4487 secs
Laptop 2 (Remote)	Home Ethernet (EduVPN)	100 queries	31.0700 min	19.8319 secs
Laptop 2 (Remote)	Home Ethernet (EduVPN)	100 queries	30.3668 min	19.3831 secs
Laptop 2 (Remote)	University Wi-Fi (Eduroam)	50 queries	14.8652 min	18.9768 secs
Laptop 2 (Remote)	University Wi-Fi (Eduroam)	50 queries	9.9379 min	12.6866 secs
Laptop 2 (Remote)	University Wi-Fi (Eduroam)	50 queries	9.7330 min	12.4251 secs
Laptop 2 (Remote)	University Wi-Fi (Eduroam)	100 queries	28.7361 min	18.3422 secs
Laptop 2 (Remote)	University Wi-Fi (Eduroam)	100 queries	27.6318 min	17.6373 secs
Laptop 2 (Remote)	University Wi-Fi (Eduroam)	100 queries	27.7121 min	17.6886 secs
Laptop 2 (Remote)	University Ethernet	50 queries	3.7039 min	4.7284 secs
Laptop 2 (Remote)	University Ethernet	50 queries	3.5372 min	4.5156 secs
Laptop 2 (Remote)	University Ethernet	50 queries	3.5162 min	4.4888 secs
Laptop 2 (Remote)	University Ethernet	100 queries	6.7794 min	4.3273 secs
Laptop 2 (Remote)	University Ethernet	100 queries	6.7416 min	4.3031 secs
Laptop 2 (Remote)	University Ethernet	100 queries	6.7598 min	4.3148 secs

Table A.1: Wall Times for Remote Query Tests Across Devices and Networks with Connection Conditions.

NB:

Average was computed using completed queries, skipped queries where no such term was found was not taken into consideration.

$(time/(Qtotal - Qskipped) = avgtime)$